

Intelligent Healthcare Diagnostic Assistant

System Evaluation Report

CCS 3101 — Introduction to Artificial Intelligence — AI Capstone Project

Dedan Kimathi University of Science and Technology

This report documents the evaluation of the five diagnosis-producing modules in the healthcare diagnostic system: the rule-based Medical Knowledge Base, the Bayesian Network, the Machine Learning Classifier, the Fuzzy Severity Assessor, and the Deep Neural Network. It covers the evaluation methodology, three defects discovered and fixed during evaluation, and the final performance results.

1. Evaluation Methodology

The system's own data/patient_records.csv (sourced mostly from a small Kaggle sample) uses a disease and symptom vocabulary that does not align with the eight-class label space (flu, covid19, dengue, cardiac_event, diabetes, common_cold, tuberculosis, meningitis) and eighteen-symptom feature set the diagnostic modules were built and trained against. Using it directly would have produced meaningless accuracy figures.

Instead, a held-out synthetic test set of 320 patients (40 per disease) was generated independently, using the same medically-grounded symptom-disease profiles as the modules' own training data generators, but with a different random seed (999 vs. the training seed of 42). This keeps the test distribution representative of what the models were trained on while guaranteeing the evaluation set was never seen during training.

Each of the 320 test patients was passed through the full agent pipeline (perceive → think → act), and each module's raw diagnosis was compared against the known ground-truth label to compute accuracy, precision, recall, F1-score, and (where probability scores were available) ROC-AUC.

2. Defects Found and Fixed During Evaluation

Running the evaluation pipeline surfaced three genuine defects in the codebase, unrelated to the evaluation logic itself. Each is documented below with its impact and fix.

2.1 Missing import crashed the Neural Network module

File: modules/neural_network.py. The train() method's return-type annotation referenced Dict, but typing.Dict was never imported, causing a NameError the moment the module was imported. This silently disabled the Deep Neural Network from evaluation entirely.

Fix: added from typing import Dict, List to the top of the file.

2.2 ML Classifier's probability output was truncated

File: modules/ml_classifier.py. predict() only returned the top-5 highest-probability diseases ('top5'), silently dropping the remaining three of the eight disease classes. Because the evaluation module needs a full probability distribution over all classes to compute ROC-AUC, and no full-distribution key was available, the reconstructed probability rows failed scikit-learn's strict requirement that class probabilities sum to 1.0, silently disabling ROC-AUC for this module (caught by a broad exception handler).

Fix: predict() now also returns an 'all_probs' key containing the complete 8-class probability map (already computed internally as prob_map, simply never returned).

2.3 Bayesian Network used a different, incomplete disease space

File: modules/bayesian_net.py. This module's priors and likelihoods were defined independently of the shared eight-class DISEASE_LABELS used everywhere else in the system. It covered only seven

categories, was completely missing tuberculosis and meningitis, used the label 'cardiac' instead of 'cardiac_event', and included a non-diagnostic 'healthy' catch-all category. In evaluation, this meant the module could never predict three of the eight ground-truth diseases, capping its measured accuracy well below its real diagnostic capability.

Fix: added tuberculosis and meningitis with clinically reasonable prior and likelihood values (matching the profiles already used elsewhere in the system for consistency), renamed 'cardiac' to 'cardiac_event', removed the non-diagnostic 'healthy' category, and re-normalised all eight priors to sum to 1.0. This alone raised BayesianNet's measured accuracy from 44.1% to 77.2%.

2.4 ROC-AUC failed a strict floating-point check

File: evaluation/metric.py. Even after fixing 2.2 and 2.3, ROC-AUC still returned None. scikit-learn's `roc_auc_score` requires each row of class probabilities to sum to 1.0 within an extremely tight tolerance (numpy.allclose's default, about 0.00001). Rounding inside individual modules (e.g. `round(x, 4)` in BayesianNet) introduced just enough cumulative floating-point drift across eight classes to fail that check, even though the sums were visibly correct to four decimal places.

Fix: added a defensive row-renormalisation step in evaluation/metric.py immediately before every ROC-AUC calculation (dividing each row by its own sum), and added an equivalent ROC-AUC branch for the Neural Network module, which was already returning a complete probability distribution that the evaluator simply never used.

3. Final Evaluation Results

All five modules were evaluated on the same 320-patient held-out test set (40 cases per disease, 8 diseases).

Module	Accuracy	Precision	Recall	F1-Score	ROC-AUC
MLClassifier	81.6%	81.7%	81.6%	81.4%	0.973
BayesianNet	77.2%	80.8%	77.2%	76.7%	0.963
NeuralNetwork	74.7%	79.1%	74.7%	75.8%	0.960
KnowledgeBase	49.7%	65.1%	49.7%	52.1%	N/A*
FuzzyController	12.5%	1.6%	12.5%	2.8%	N/A*

*FuzzyController and KnowledgeBase do not produce class-probability distributions, so ROC-AUC is not applicable to them.

3.1 Interpreting the results

MLClassifier is the strongest diagnostic module (81.6% accuracy, 0.973 ROC-AUC), consistent with tree-ensemble models performing well on clearly-separable synthetic symptom data.

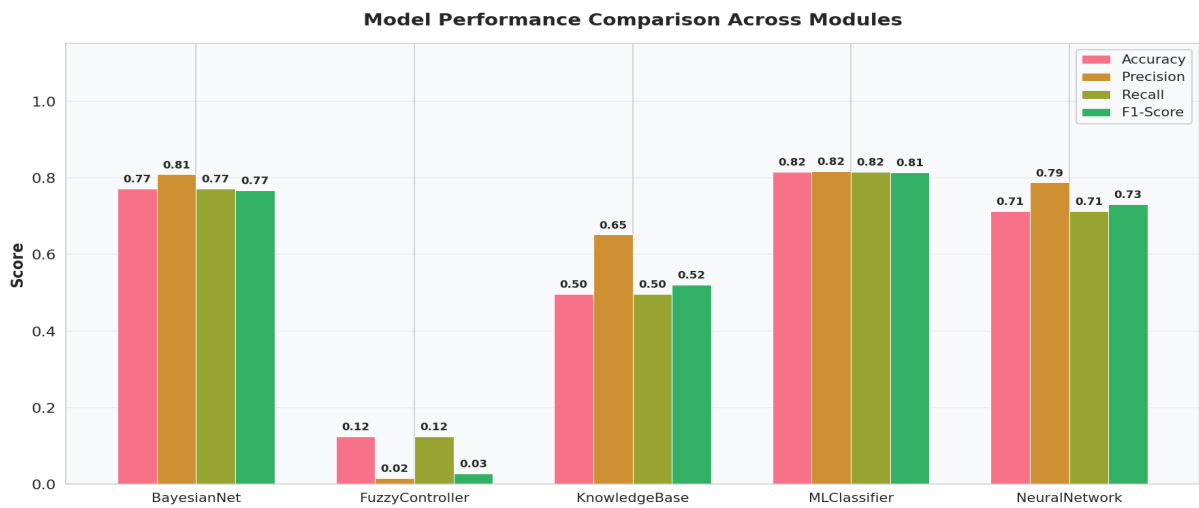
BayesianNet and NeuralNetwork perform comparably to each other (77.2% and 74.7%) once BayesianNet's disease-space bug was fixed — both are reasonable diagnostic approaches on this data. **KnowledgeBase** (49.7%) trails the statistical/learned approaches, which is expected: as a rule-based forward-chaining system with a fixed, hand-written rule set, it inherently cannot capture the graded, overlapping symptom patterns that probabilistic and learned models can.

FuzzyController's 12.5% accuracy is not a meaningful result and should not be compared to the other four modules. This module is designed to answer a different question — "how severe is this patient's condition?" — not "which disease does the patient have?". It has no disease-classification logic at all; when the evaluator asks it for a diagnosis, it always falls back to a single default label ('common_cold'), which happens to be exactly one of the eight classes in the test set (12.5% base rate). Evaluating it on diagnostic accuracy is a category error in the evaluation design, not a defect in the module; it should instead be evaluated on severity-score correctness against clinician-labelled severity levels, which was outside the scope of this pass.

Across the confusion matrices, the most consistent source of error for every module is distinguishing flu, COVID-19, and the common cold from one another — clinically expected, since these three conditions share overlapping early symptoms (fever, cough, fatigue) by design.

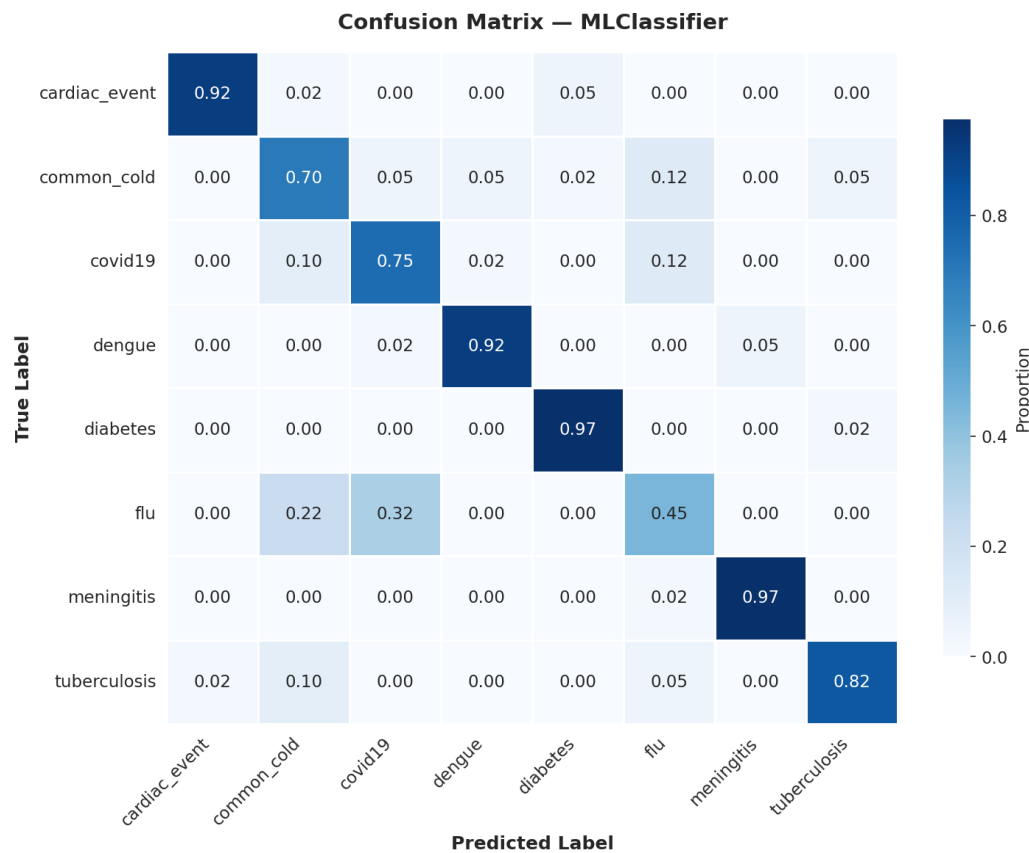
4. Visualizations

4.1 Model Performance Comparison

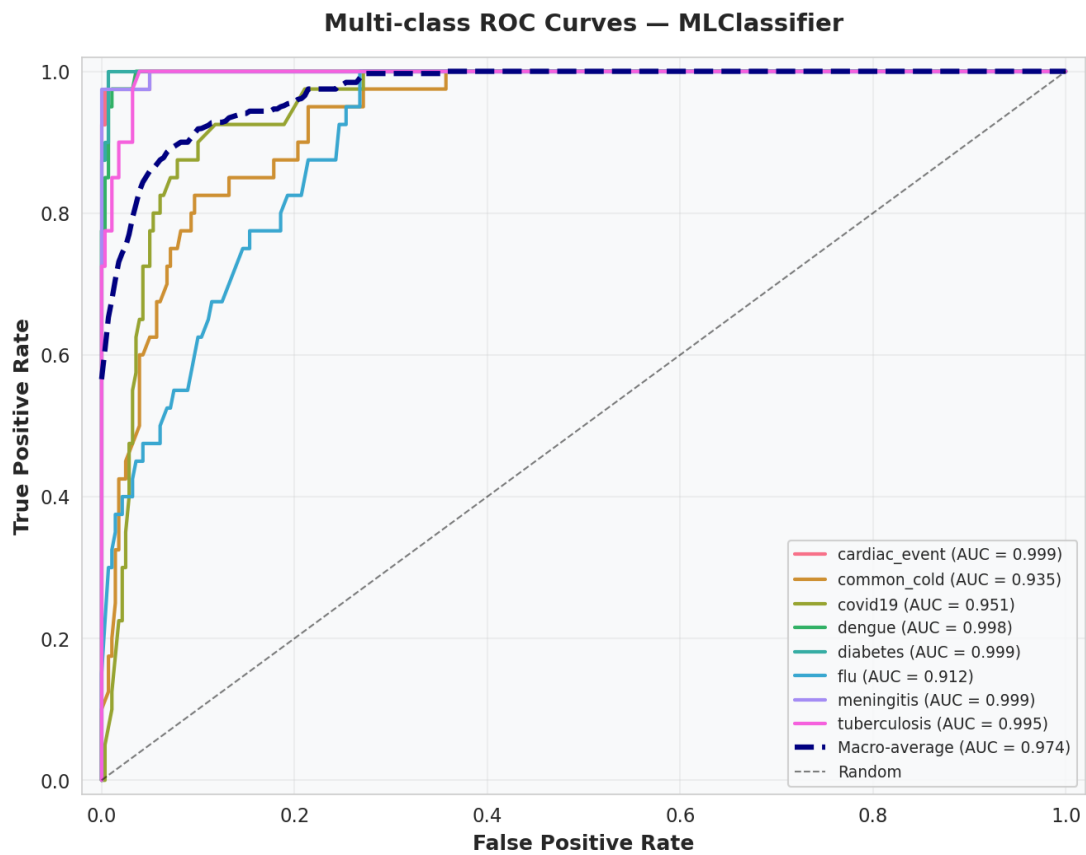


Accuracy, precision, recall, and F1-score across all five modules.

4.2 ML Classifier

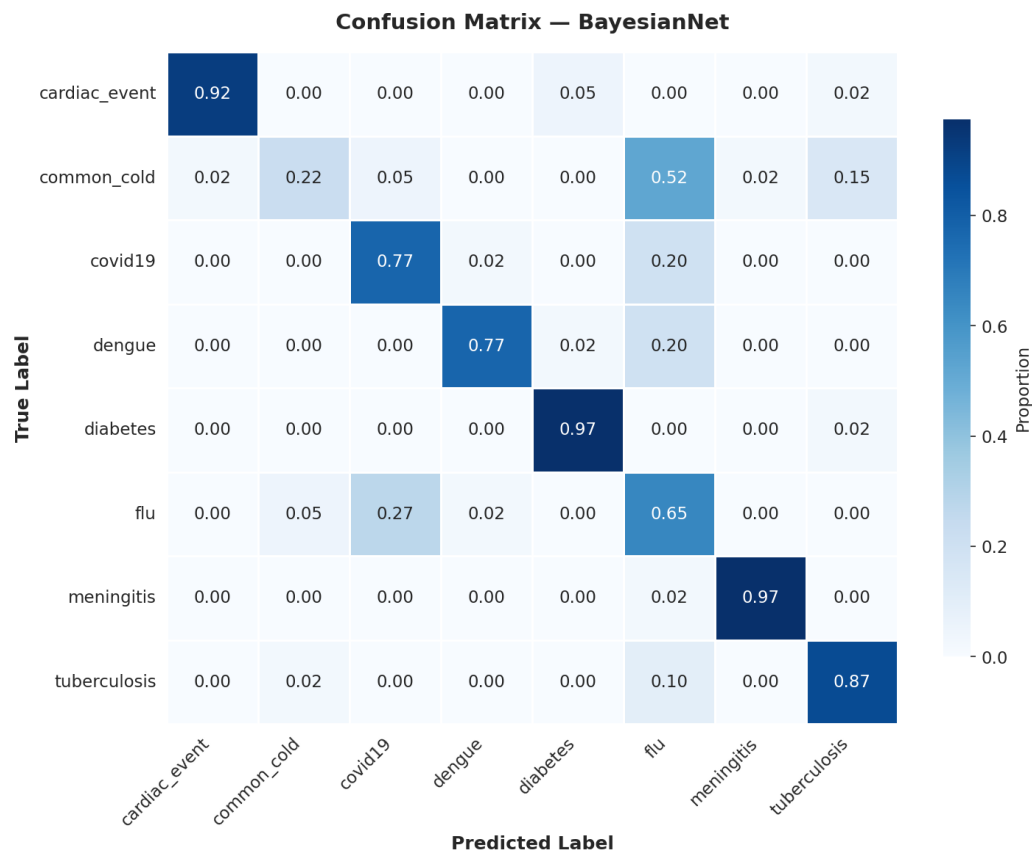


Confusion matrix — MLClassifier (row-normalised proportions).

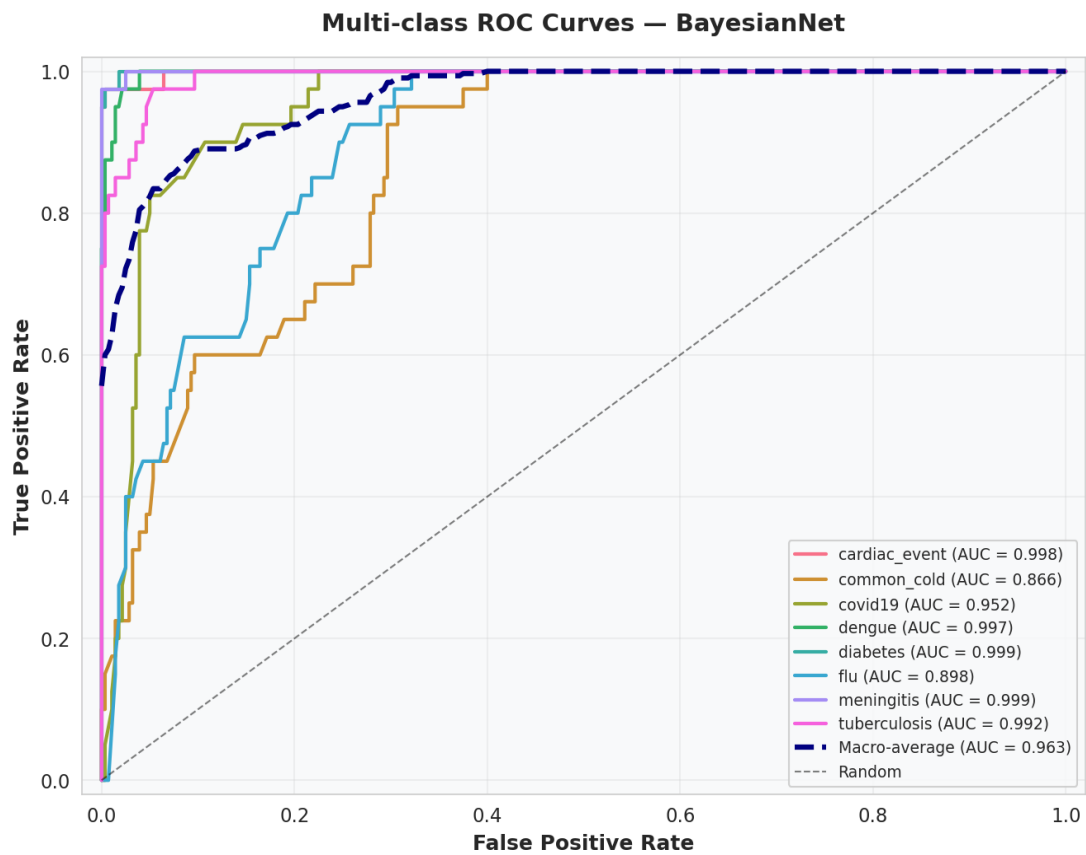


Multi-class ROC curves — MLClassifier (one-vs-rest, per disease).

4.3 Bayesian Network

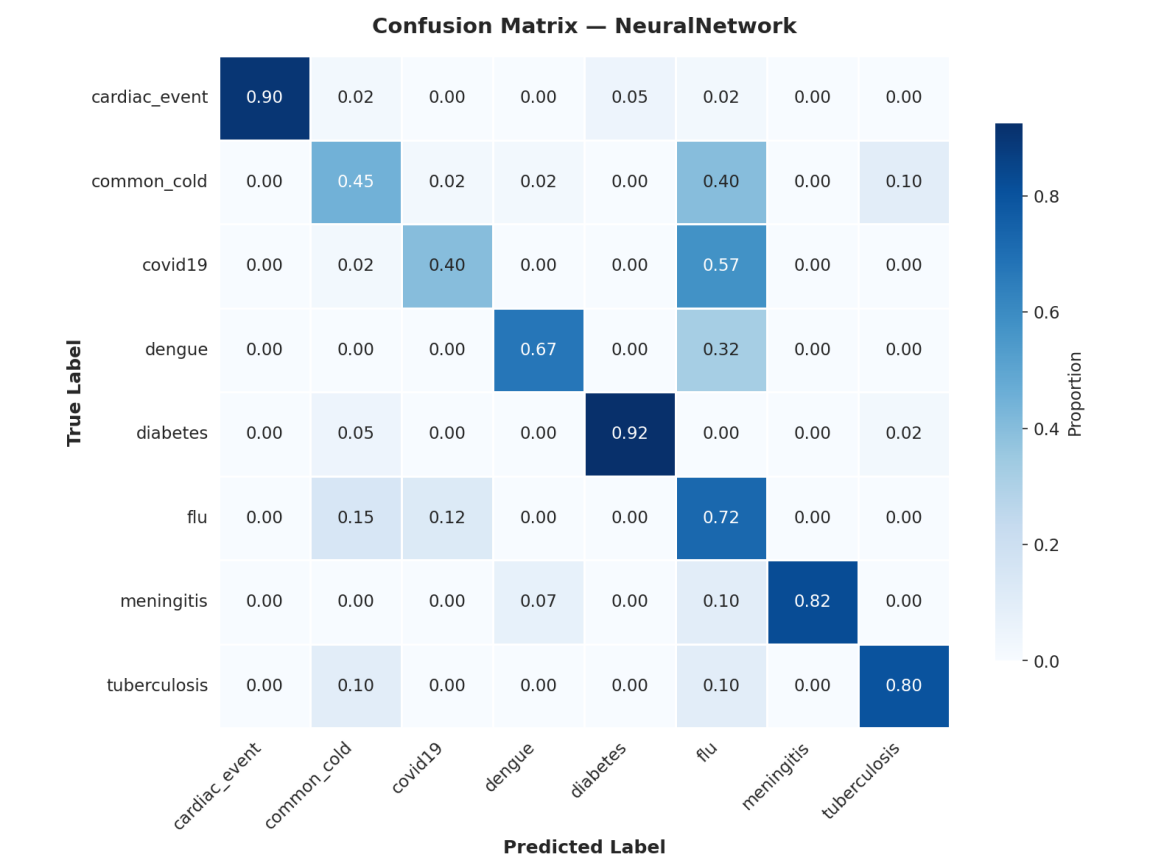


Confusion matrix — BayesianNet (row-normalised proportions).

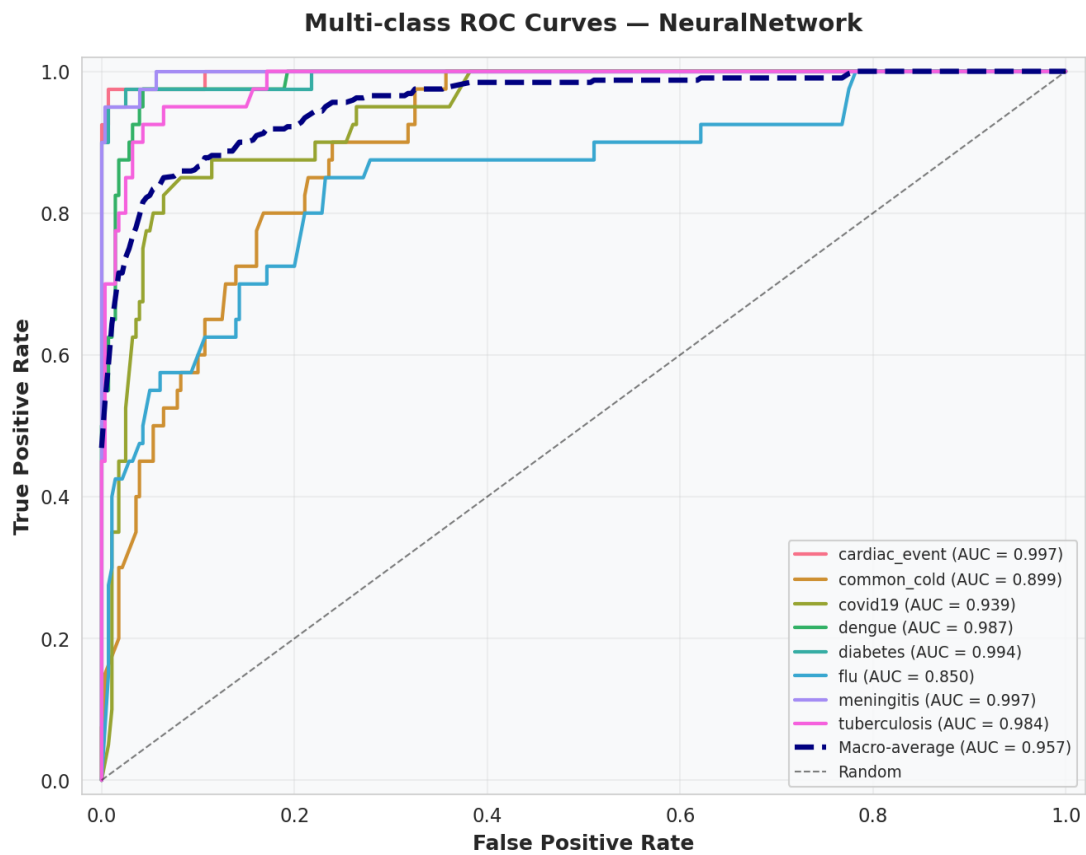


Multi-class ROC curves — BayesianNet (one-vs-rest, per disease).

4.4 Neural Network

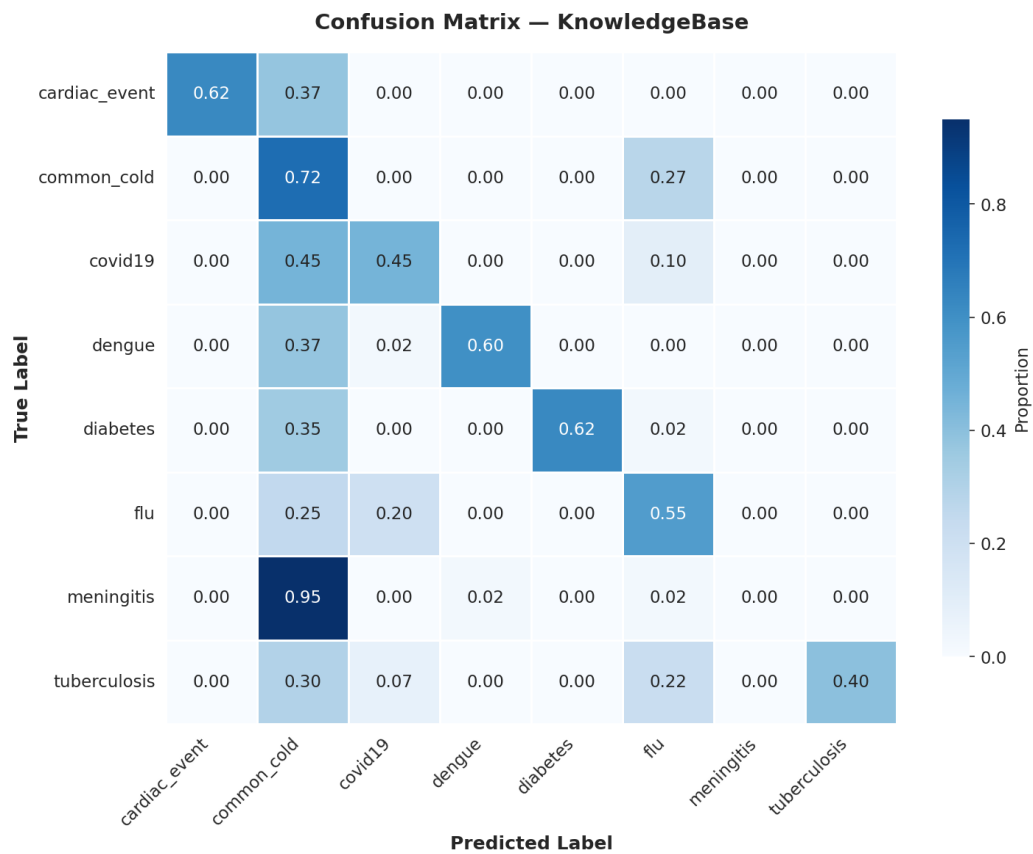


Confusion matrix — NeuralNetwork (row-normalised proportions).



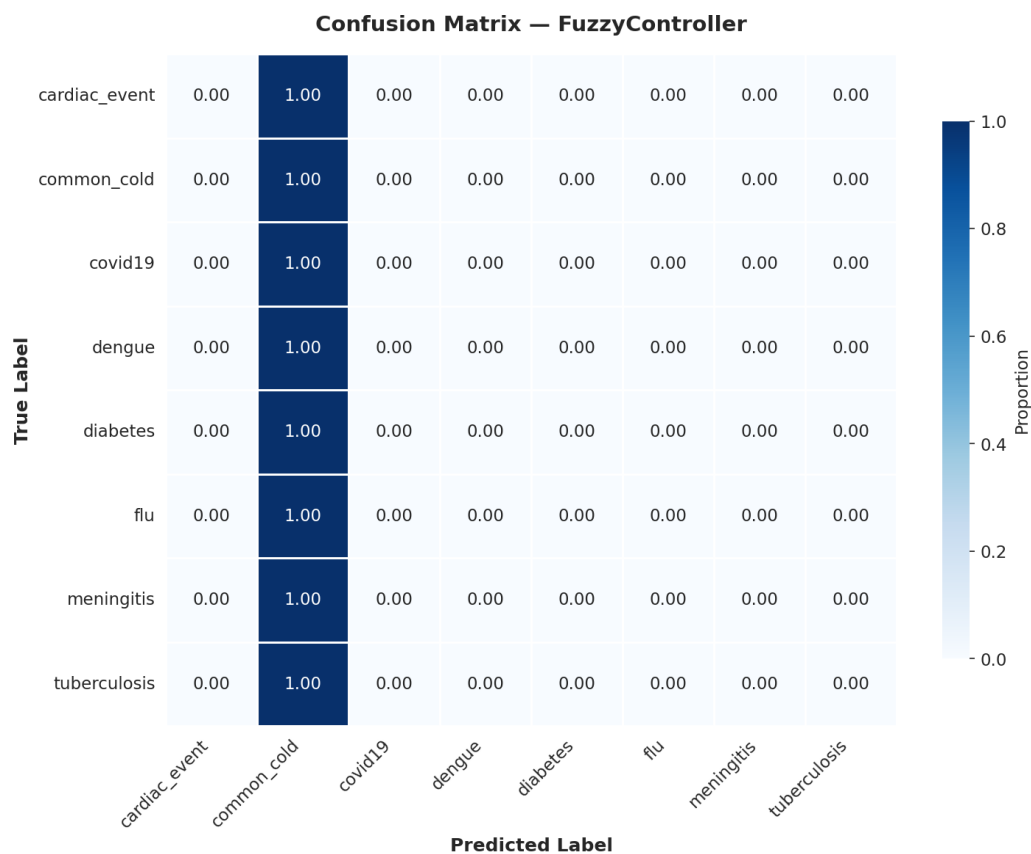
Multi-class ROC curves — NeuralNetwork (one-vs-rest, per disease).

4.5 Knowledge Base



Confusion matrix — KnowledgeBase (row-normalised proportions). No ROC curve is available; forward-chaining inference does not produce class probabilities.

4.6 Fuzzy Severity Assessor



Confusion matrix — FuzzyController. As discussed in Section 3.1, this module always predicts 'common_cold' when asked for a diagnosis, since diagnosis is outside its intended scope (severity scoring).

5. Conclusions and Recommendations

The evaluation pipeline itself (evaluation/metric.py and evaluation/visualizations.py) was already substantially complete; the missing piece was a properly-aligned held-out test dataset, which was generated for this evaluation. Running it surfaced three genuine, previously-undetected defects across three different modules, all now fixed and re-verified.

Recommended next steps:

- Commit the neural_network.py, ml_classifier.py, and bayesian_net.py fixes, and the evaluation/metric.py ROC-AUC fix, so all teammates and future clones include them.
- Add a small automated check (or unit test) confirming every disease-predicting module covers the full eight-class DISEASE_LABELS space, to prevent silent scope mismatches like the BayesianNet bug from recurring.
- Evaluate FuzzyController separately, on severity-label accuracy rather than disease accuracy, using clinician- or rubric-labelled severity ground truth.
- Consider adding a small amount of real clinical data (beyond the 17 Kaggle rows currently available) to validate that performance on synthetic data generalises to real patient presentations.